

KZG Polynomial Commitment scheme

Pushpendra Pal

Abstract

Polynomial Commitment Schemes. Roughly speaking, a polynomial commitment scheme is simply a commitment scheme in which the **object being committed to is (all evaluations of) a low-degree polynomial**. That is, a polynomial commitment scheme allows a prover to commit to a polynomial $\tilde{w}$ satisfying a specified degree bound, and later **reveal $\tilde{w}(r)$ for a point r of the verifier's choosing**. Even though in the commitment phase the prover does not send all evaluations of $\tilde{w}$ to the verifier, the commitment still effectively binds the prover. That is, at a later time, the verifier can ask the prover to reveal $\tilde{w}(r)$ for any desired r of the verifier's choosing, and the prover is effectively forced to reveal $\tilde{w}(r)$ for a fixed polynomial $\tilde{w}$ determined at the time of the original commitment. In particular, **the prover is unable to choose the polynomial $\tilde{w}$ to depend on the query point r** , at least not without breaking the computational assumption on which security of the commitment scheme is based.

1 PCS

1.1 Setup

- Three Groups G_1 , G_2 and G_T . G_1 and G_2 are additive, G_T is multiplicative. (Read the pairing over Elliptic curves)
- $e(G_1, G_2) \rightarrow G_T$ is a bilinear pairing.
- $\alpha \xleftarrow{R} \mathbb{F}_p$, α is a random element, should be deleted. This is why KZG has trusted setup.
- **SRS**

$$\mathbb{H} = gp_1 = \{h_0 = \alpha^0 g_1, h_1 = \alpha^1 g_1, \dots, h_{d-1} = \alpha^{d-1} g_1\} \quad (1)$$

$$= \{h_0 = g_1, h_1 = \alpha g_1, h_2 = \alpha^2 g_1, \dots, h_{d-1} = \alpha^{d-1} g_1\} \quad (2)$$

$$gp_2 = \{g_2, \alpha g_2\} \quad (3)$$

Here, d = degree of the polynomial to be committed. Should be higher because the polynomial to be committed is not known at this stage.

1.2 Commitment ($gp_1, P(x)$)

- $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_{d-1}x^{d-1}$, where $a_i \in \mathbb{F}_p$
- $P(\alpha) = ?$, because α is deleted.
- Use $\mathbb{H}$

$$g_1 P(\alpha) = h_0 a_0 + h_1 a_1 + h_2 a_2 + \dots + h_{d-1} a_{d-1}$$

$$g_1 P(\alpha) = g_1 a_0 + \alpha g_1 a_1 + \alpha^2 g_1 a_2 + \dots + \alpha^{d-1} g_1 a_{d-1}$$

- $com_P = g_1 P(\alpha)$, where g_1 is the generator of G_1 .
- Note that $com_P = g_1 P(\alpha) \in G_1$, which is important for pairing.

1.3 Opening ($com_P, com_Q, r, P(r)$)

Described in the protocolview

1.4 Protocol view

Committer ($P(x), gp_1,$ gp_2)	$\xleftarrow{gp_1, gp_2}$ Setup(α) $\xrightarrow{gp_1, gp_2}$	Verifier (gp_1, gp_2)

$$com_P = g_1 P(\alpha)$$

$$\begin{array}{c} \longrightarrow \xrightarrow{com_P} \longrightarrow \\ \longleftarrow \xleftarrow{r} \longleftarrow \end{array}$$

$$r \xleftarrow{R} \mathbb{F}_p$$

- Calc $P(r) = v$
- Define $Q(x)$:

$$= \frac{P(x) - P(r)}{x - r}$$

$$= \frac{P(x) - v}{x - r}$$
- Calc $com_Q = g_1 Q(\alpha)$
the same way as before for com_P .
- And that would be equal to:

$$com_Q = \frac{g_1 P(\alpha) - g_1 v}{\alpha - r}$$

$$\longrightarrow \xrightarrow{com_Q, v} \longrightarrow$$

- Calc $g_1 v$. And we already have $\alpha g_2 \in gp_2$
- Check

$$e[com_P - g_1 v, g_2]$$

$$=? e[com_Q, g_2 \alpha - g_2 r]$$
- Since $g_2 \alpha$ is known although α is not, we can calc $g_2 r$

1.5 Notes

1. Final derivation:

$$\begin{aligned}
 &= e \left[com_P - g_1 v, g_2 \right] = e \left[g_1 P(\alpha) - g_1 v, g_2 \right] \quad \dots \text{Verifier calcs } g_1 v \text{ and } g_1 P(\alpha) = com_P; \\
 &= e \left[(\alpha - r) com_Q, g_2 \right] \quad \dots \dots \dots com_Q = \frac{g_1 P(\alpha) - g_1 v}{\alpha - r} \\
 &= e \left[com_Q, g_2 (\alpha - r) \right] \quad \dots \dots \dots e(aP, Q) = e(P, aQ) \text{ (bilinear pairing)} \\
 &= e \left[com_Q, g_2 \alpha - g_2 r \right] \quad \dots \dots \dots \text{Verifier calcs } g_2 r
 \end{aligned}$$

2. The values $P(\alpha)$, v , r and α are all scalars. Important becoz G_1, G_2 are additive groups.

3. In a Merkle hash tree, a polynomial is committed using all its values. The prover has to evaluate the polynomial to be committed at every point and then form the merkle hash tree. Instead in KZG, the prover commits the polynomial itself as in the coefficients (in a way) and the verifier can find the evaluation at any point r of his choosing to open the commitment.

4. Suppose Prover knows α , and he takes another polynomial $P'(x)$, such that $P(\alpha) = P'(\alpha)$ then he uses $P'(x)$ to find $Q'(x) = \frac{P'(x) - P'(r)}{x - r}$, and also calculates $v' = P'(r)$. Hence opening the commitment to a different value v' .

5. **Knowledge of Exponent Assumption KEA in pairing based groups:** Read about this.

6. PROS:

- Open requires just two elements to be sent by the prover: com_Q and v .
- Batch proof possible becoz of additive homomorphism of commitments, $com_{P_1} + com_{P_2} = com_{P_1 + P_2}$.

7. CONS:

- Trusted setup needed, as in the setup must forget α and there should be no way to derive it. Otherwise the prover may cheat. **Study DORY PCS.**